

tf.constant Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/constant

Creates a constant tensor from a tensor-like object.

Function Signatures

```
tf.constant( value, dtype=None, shape=None, name='Const' ) -> Union[tf.Operation, ops._EagerTensorBase]
```

Code Examples

```
tf.constant(  
value, dtype=None, shape=None, name='Const'  
) -> Union[tf.Operation, ops._EagerTensorBase]
```

```
tf.constant(  
value, dtype=None, shape=None, name='Const'  
) -> Union[tf.Operation, ops._EagerTensorBase]
```

```
tf.Operation
```

```
# Constant 1-D Tensor from a python list.  
tf.constant([1, 2, 3, 4, 5, 6])  
  
# Or a numpy array  
a = np.array([[1, 2, 3], [4, 5, 6]])  
tf.constant(a)
```

```
# Constant 1-D Tensor from a python list.
```

```
tf.constant([1, 2, 3, 4, 5, 6])
```

```
numpy=array([1, 2, 3, 4, 5, 6], dtype=int32)>
```

Documentation

Creates a constant tensor from a tensor-like object.

Used in the notebooks

- Better performance with `tf.function`
 - TF-NumPy Type Promotion
 - Introduction to Tensors
 - Introduction to graphs and `tf.function`

- Migrate `tf.feature_column`'s` to Keras preprocessing layers
- DeepDream
- Load text
- Playing CartPole with the Actor-Critic method
- Neural style transfer
- Custom training loop with Keras and `MultiWorkerMirroredStrategy`

If the argument `dtype` is not specified, then the type is inferred from the type of value.

If `dtype` is specified, the resulting tensor values are cast to the requested `dtype`.

If `shape` is set, the value is reshaped to match. Scalars are expanded to fill the shape:

`tf.constant` has no effect if an eager Tensor is passed as the value, it even transmits gradients:

But, since `tf.constant` embeds the value in the `tf.Graph` this fails for symbolic tensors:

`tf.constant` will create tensors on the current device. Inputs which are already tensors maintain their placements unchanged.

Related Ops:

- `tf.convert_to_tensor` is similar but:

It has no `shape` argument.

Symbolic tensors are allowed to pass through.

```
with tf.compat.v1.Graph().as_default():
```

```
i = tf.compat.v1.placeholder(shape=[None, None], dtype=tf.float32)
```

```
t = tf.convert_to_tensor(i)
```

- It has no shape argument.
- Symbolic tensors are allowed to pass through.
- `tf.fill`: differs in a few ways:

`tf.constant` supports arbitrary constants, not just uniform scalar

Tensors like `tf.fill`.

`tf.fill` creates an Op in the graph that is expanded at runtime, so it can efficiently represent large tensors.

Since `tf.fill` does not embed the value, it can produce dynamically sized outputs.

- `tf.constant` supports arbitrary constants, not just uniform scalar

Tensors like `tf.fill`.

- `tf.fill` creates an Op in the graph that is expanded at runtime, so it

can efficiently represent large tensors.

- Since `tf.fill` does not embed the value, it can produce dynamically

sized outputs.

`tf.convert_to_tensor` is similar but:

- It has no shape argument.

- Symbolic tensors are allowed to pass through.

`tf.fill`: differs in a few ways:

- `tf.constant` supports arbitrary constants, not just uniform scalar

Tensors like `tf.fill`.

- `tf.fill` creates an Op in the graph that is expanded at runtime, so it

can efficiently represent large tensors.

- Since `tf.fill` does not embed the value, it can produce dynamically

sized outputs.

Returns

Raises